Course Project: Program Testing and Coverage Analysis

bzip2.c — Fuzzing Campaign Report

Team Members:
Lena Alqaissom 202268180
Reema AlQahtani 202280860
Kawther Alkhawajah 202168810

Course: ICS 433 — Software Testing
Date: May 2026

1. Program Analysis

1.1 Overview of bzip2

bzip2 is an open-source, high-quality data compression program originally written by Julian Seward. It compresses and decompresses files using the Burrows-Wheeler block-sorting text compression algorithm combined with Huffman coding. Unlike simpler compression schemes, bzip2 achieves significantly better compression ratios at the cost of higher memory and CPU usage.

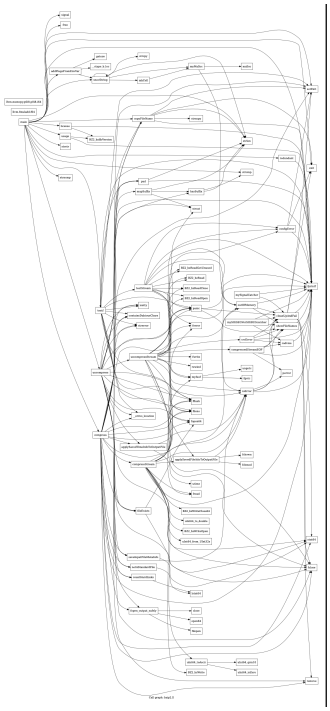
The program operates on individual files and supports three main modes of operation: compression (-z), decompression (-d), and integrity testing (-t). The source file bzip2.c provided for this project is a single amalgamated file containing approximately 7,000 lines of C code, including both the command-line frontend and the full libbzip2 library implementation.

1.2 How bzip2 Works

The compression pipeline consists of four stages: Burrows-Wheeler Transform (BWT) which rearranges data to group similar characters together, Move-To-Front (MTF) encoding, Run-Length Encoding (RLE), and Huffman coding to produce the final compressed output. Decompression reverses this pipeline exactly.

1.3 Call Graph

The call graph below illustrates the relationships between the major functions in bzip2.c. The two functions highlighted for detailed analysis are uncompressStream() at line 5416 and BZ2_bzWrite() at line 4366.



Task 1 Call Graph

2. Function Analysis

2.1 uncompressStream() — Line 5416

Purpose and Overview

uncompressStream() is the high-level stream decompression function in bzip2. It is responsible for reading a compressed .bz2 file, decompressing its contents, and writing the recovered data to an output stream. It handles multiple concatenated bzip2 streams in a single file, error conditions, and fallback behavior for non-bzip2 data.

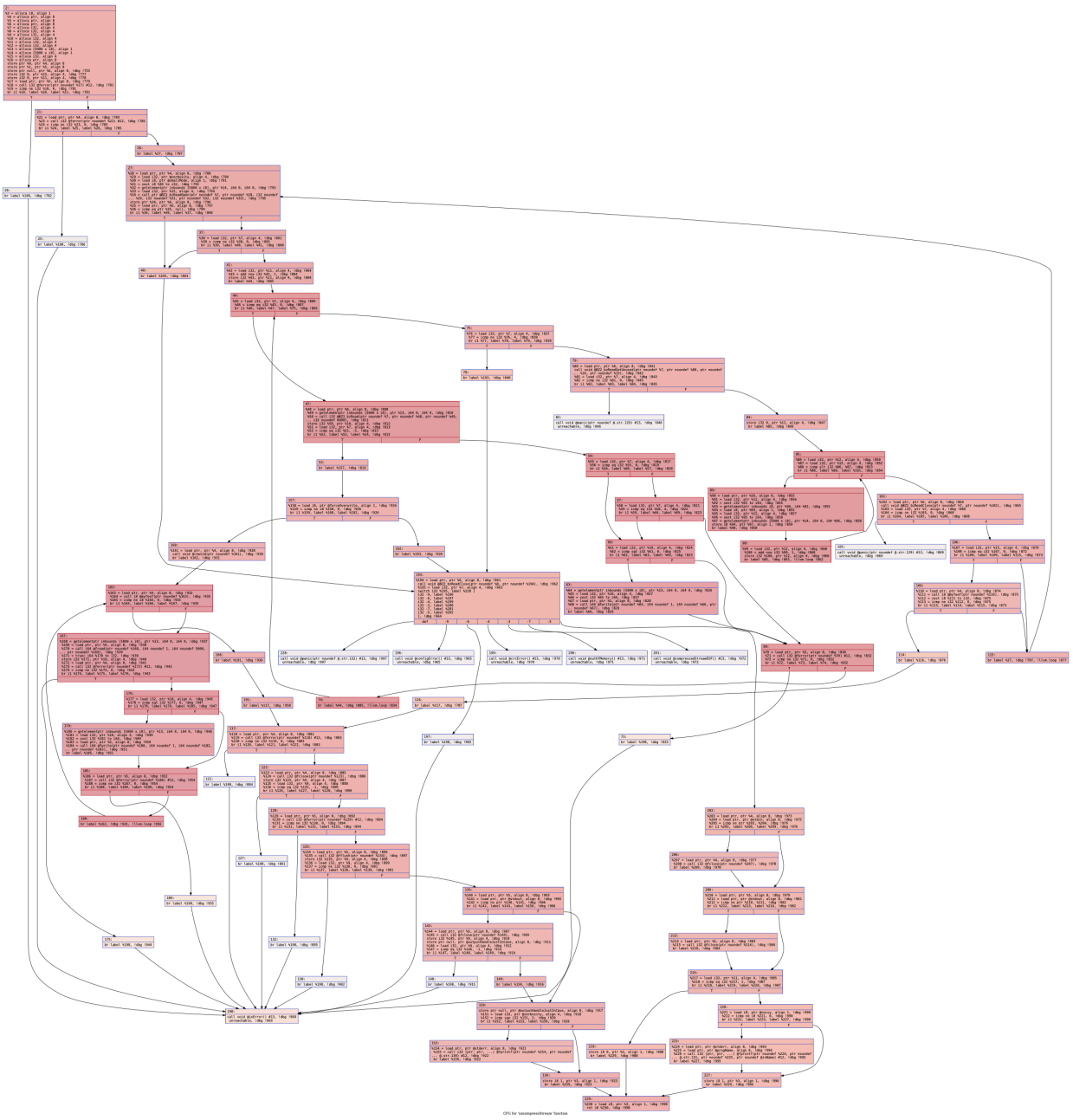
Function Signature

```
Bool uncompressStream(FILE *zStream, FILE *stream)
```

Key Steps and Logic

The function operates inside an outer while(True) loop that handles multiple concatenated bzip2 streams. For each stream it: (1) calls BZ2_bzReadOpen() to initialize the decompression engine, (2) enters an inner loop calling BZ2_bzRead() repeatedly to decompress chunks of up to 6000 bytes, (3) writes each decompressed chunk to the output stream using fwrite(), (4) calls BZ2_bzReadGetUnused() to retrieve leftover bytes, and (5) calls BZ2_bzReadClose() to clean up.

CFG — Task 1



uncompressStream_CFG — Task 1 CFG

2.2 BZ2_bzWrite() — Line 4366

Purpose and Overview

BZ2_bzWrite() is a library API function that feeds raw uncompressed data into the bzip2 compression engine. It is called repeatedly by the application to supply data for compression, and internally manages the compression loop until all provided input has been processed.

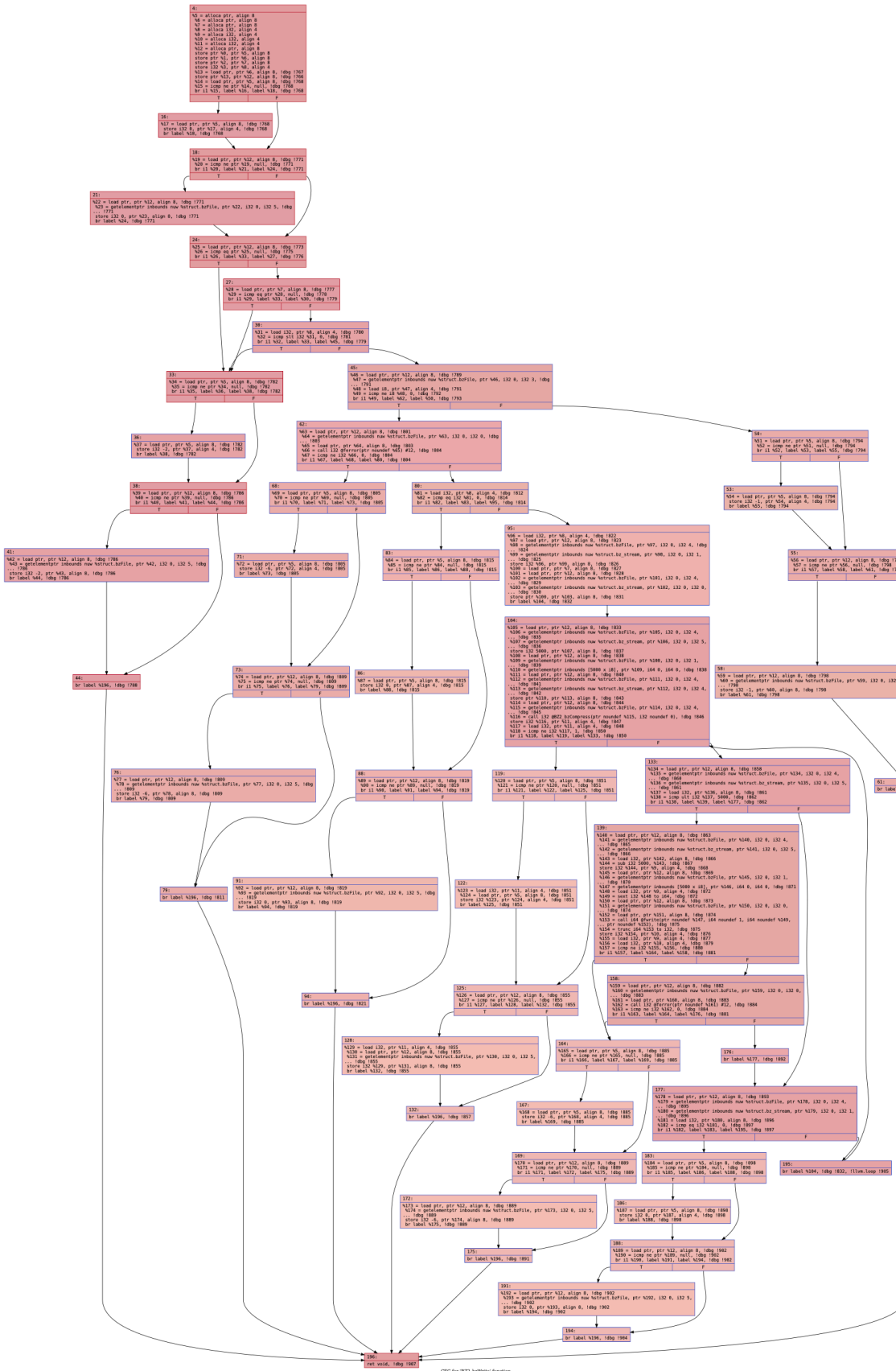
Function Signature

```
void BZ2_bzWrite(int *bzerror, BZFILE *b, void *buf, int len)
```

Key Steps and Logic

The function begins with validation checks on bzf, buf, and len parameters. After validation, it enters a while(True) loop that: (1) sets the output buffer to BZ_MAX_UNUSED bytes, (2) calls BZ2_bzCompress() with BZ_RUN action, (3) writes compressed output using fwrite(), and (4) exits when all input has been consumed (avail_in == 0).

CFG — Task 1



CFG for 'BZ2_bzWrite' function

BZ2_bzWrite_CFG — Task 1 CFG

3. Fuzzing Campaigns

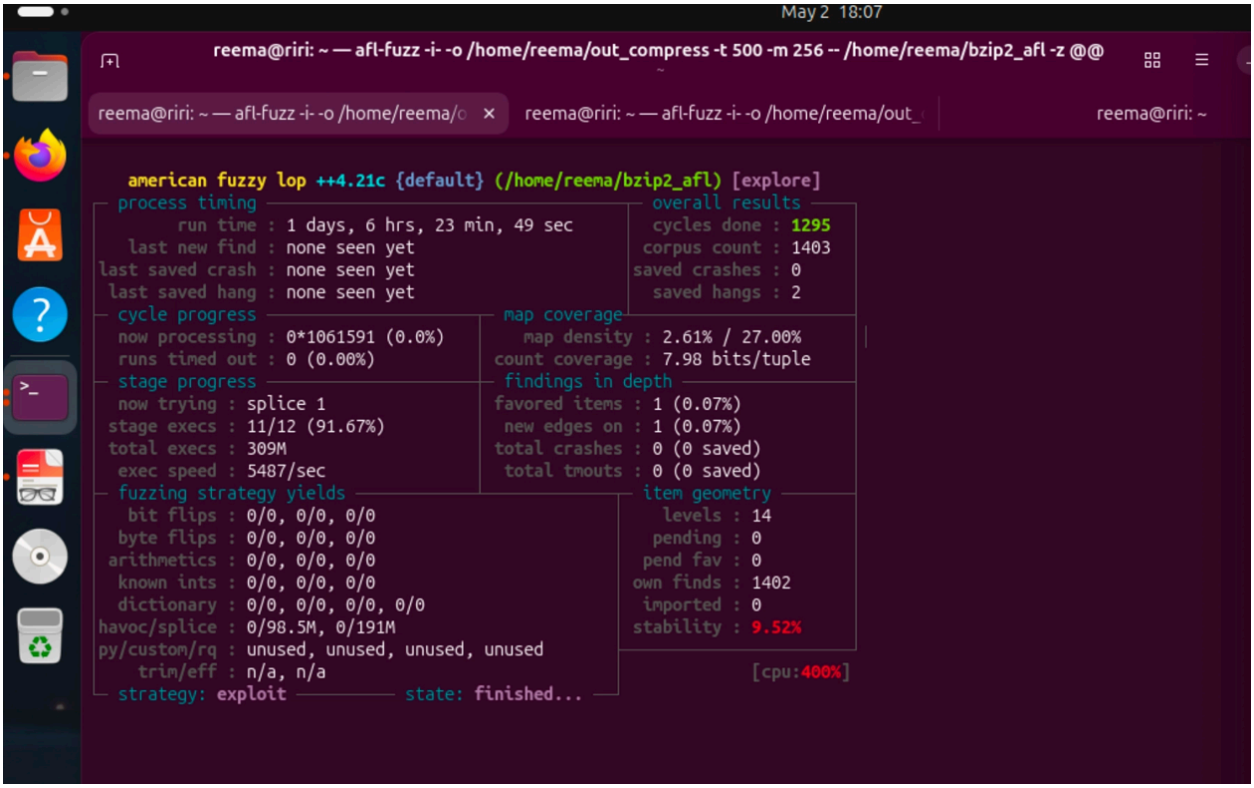
3.1 Compilation

```
afl-clang-fast -g -D_FILE_OFFSET_BITS=64 -o ~/bzip2_afl ~/bzip2.c
echo core | sudo tee /proc/sys/kernel/core_pattern
```

3.2 Campaign 1 — Compression

```
afl-fuzz -i seeds_compress -o out_compress -t 500 -m 256 -- ./bzip2_afl -z @@
```

Metric	Value
Run time	1 day, 6 hours, 23 min
Total executions	309,000,000+
Exec speed	~5,487 exec/sec
Corpus count	1,403 inputs
Saved crashes	0
Saved hangs	2



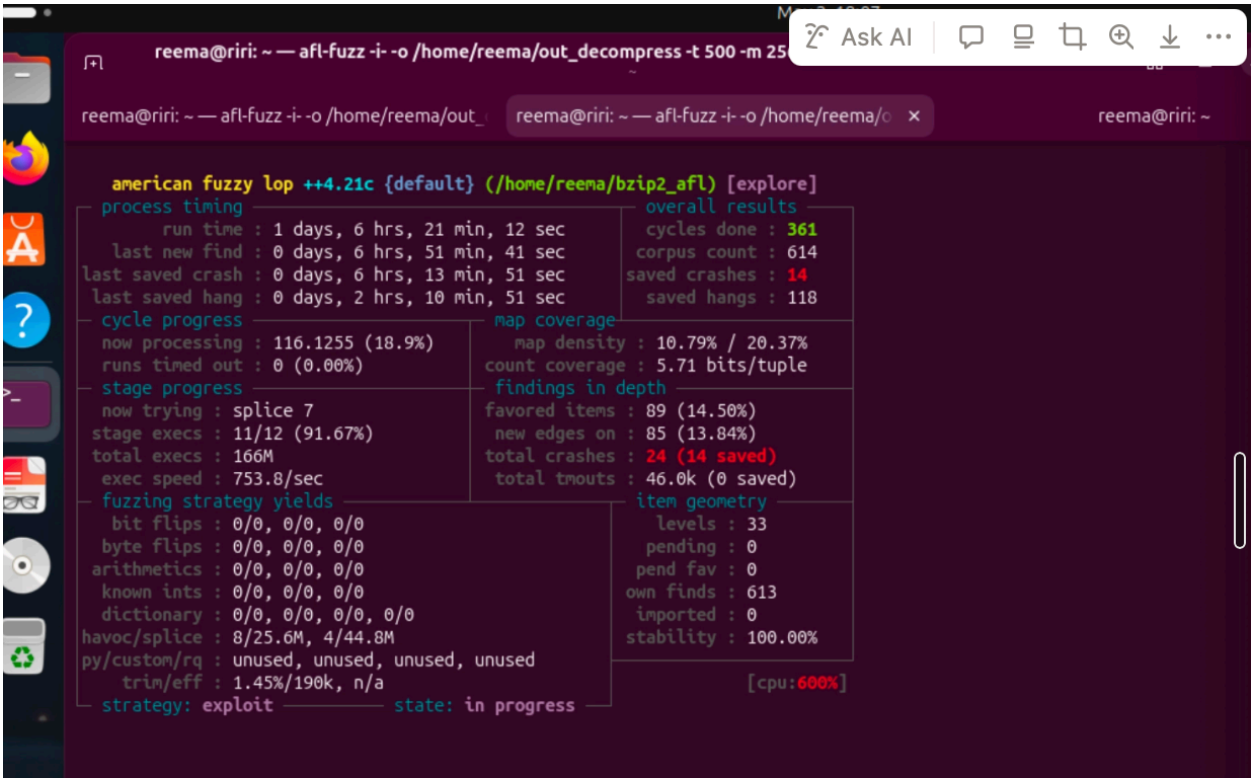
Campaign 1 final AFL++

No crashes were found during the compression campaign. This is expected since the compression path accepts any raw data as input.

3.3 Campaign 2 — Decompression

```
afl-fuzz -i seeds_decompress -o out_decompress -t 500 -m 256 -- ./bzip2_afl -d -c @@
```

Metric	Value
Run time	1 day, 6 hours, 21 min
Total executions	166,000,000+
Exec speed	~753 exec/sec
Corpus count	614 inputs
Saved crashes	14 crashes
Saved hangs	118



Campaign 2 final AFL++

3.4 Machine Specifications

Component	Details
OS	Ubuntu 24.04 LTS (ARM 64-bit, VirtualBox)
RAM	4,760 MB
AFL++ Version	4.21c
Compiler	afl-clang-fast (LLVM 20.1.8)

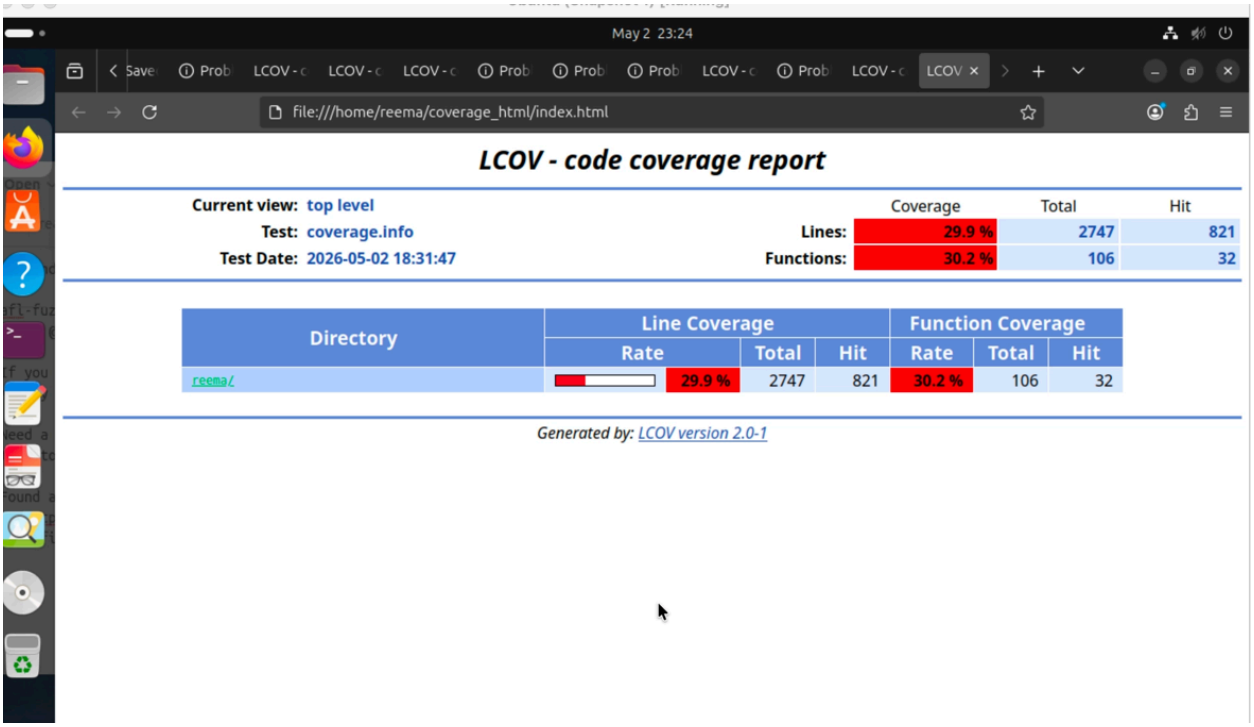
4. Source-Based Coverage

4.1 Coverage Collection

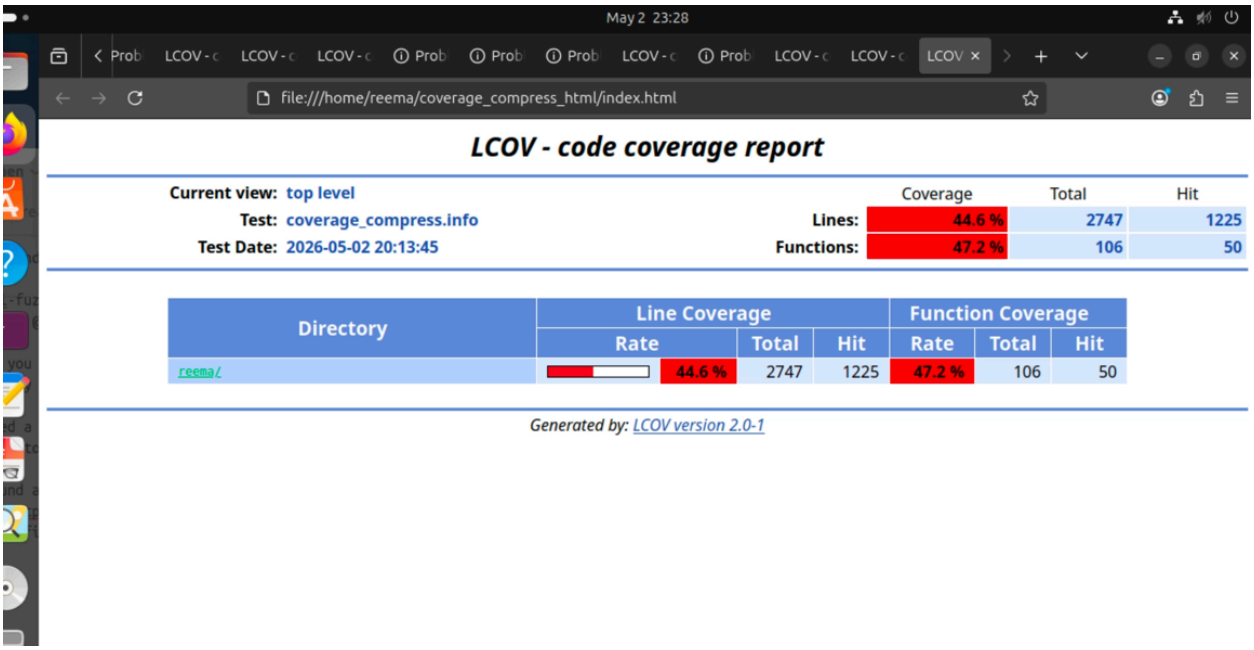
```
gcc -g -fprofile-arcs -ftest-coverage -D_FILE_OFFSET_BITS=64 -o bzip2_cov bzip2.c
-lgcov
for f in ~/out_decompress/default/queue/id:*; do ~/bzip2_cov -d -c "$f"
2>/dev/null; done
lcov --capture --directory ~ --output-file coverage.info
genhtml coverage.info --output-directory coverage_html
```

4.2 Coverage Summary

Campaign	Lines Covered	Line %	Functions Covered	Function %
Decompression	821 / 2747	29.9%	32 / 106	30.2%
Compression	1225 / 2747	44.6%	50 / 106	47.2%



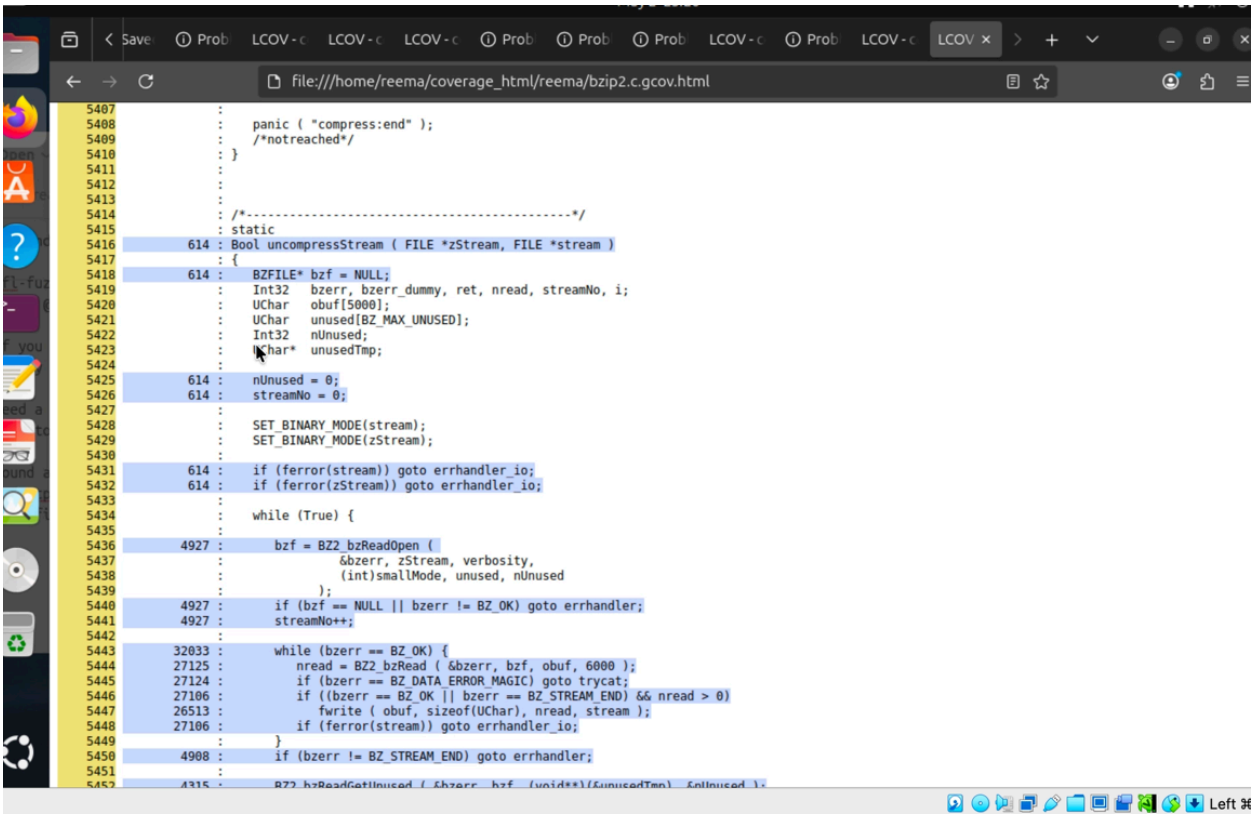
lcov Decompression Summary



Icov Compression Summary

4.3 Coverage of uncompressStream()

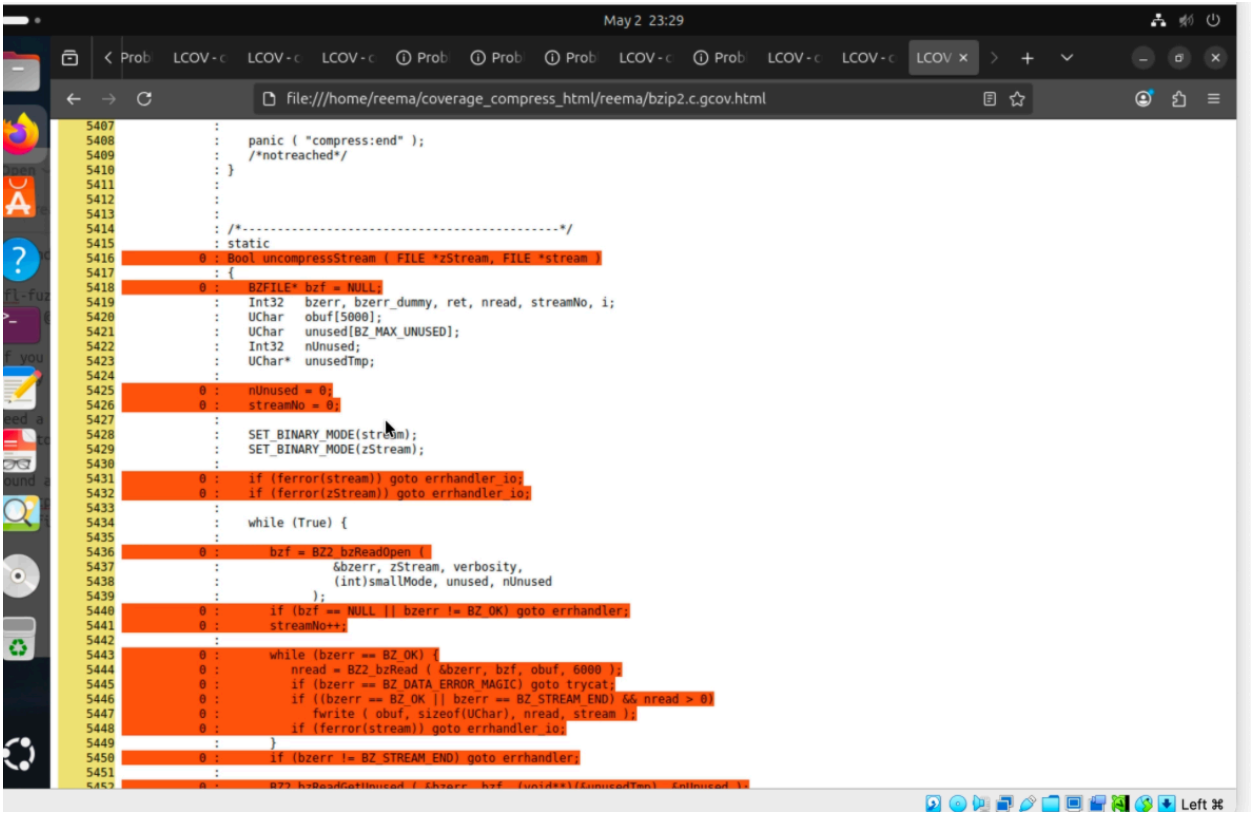
The coverage report shows that uncompressStream() at line 5416 was well-covered by the decompression fuzzing campaign. The main execution paths received thousands of hits. Error handling paths such as trycat fallback and errhandler cases were less covered as they require specific malformed input patterns.



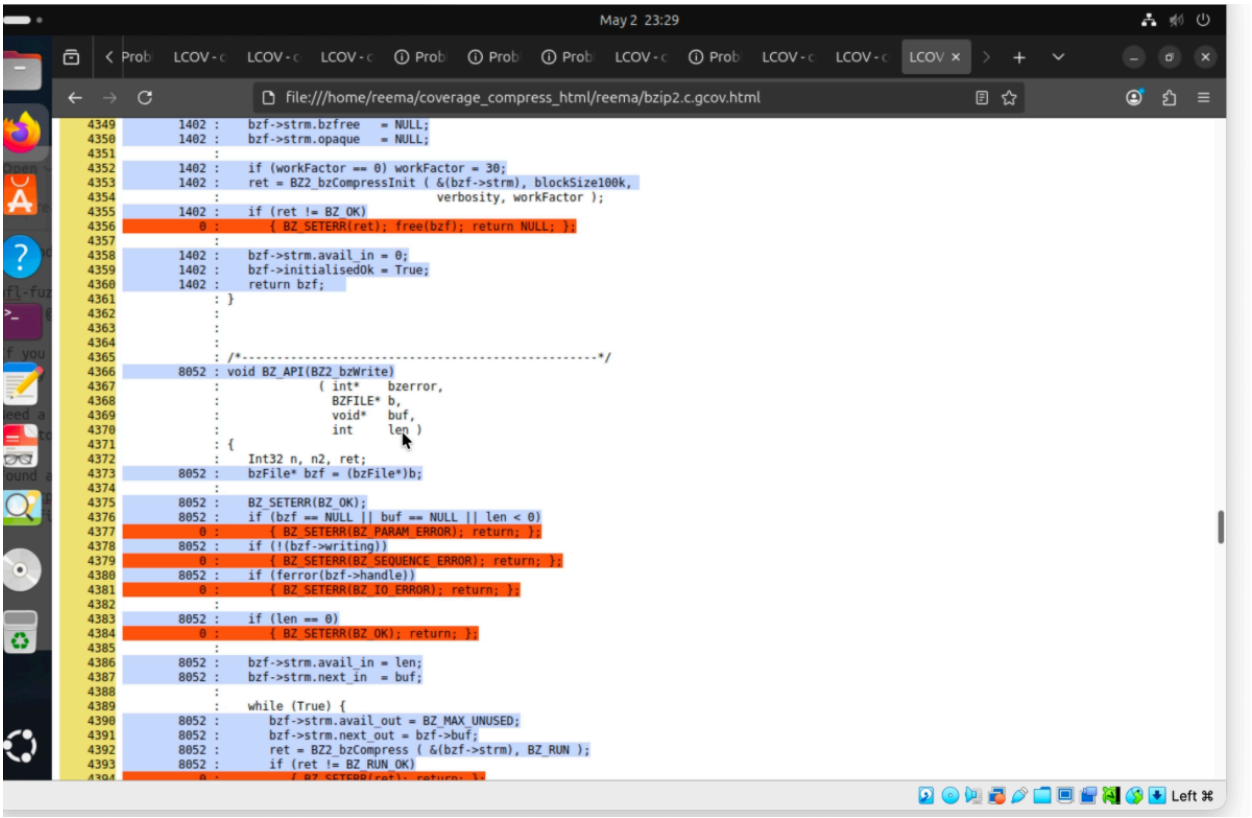
uncompressStream lcov — line 5416 (decompression campaign)

4.4 Coverage of BZ2_bzWrite()

BZ2_bzWrite() at line 4366 showed 0% coverage in the decompression campaign, which is expected since this function is only invoked during compression operations. In the compression campaign, BZ2_bzWrite() was fully covered with 8,052 hits. The uncovered portions were error return paths requiring invalid parameters or I/O failures.



BZ2_bzWrite lcov — 0 hits (decompression campaign)



BZ2_bzWrite lcov — 8,052 hits (compression campaign)

4b. Coverage Mapping and Analysis (Task 4)

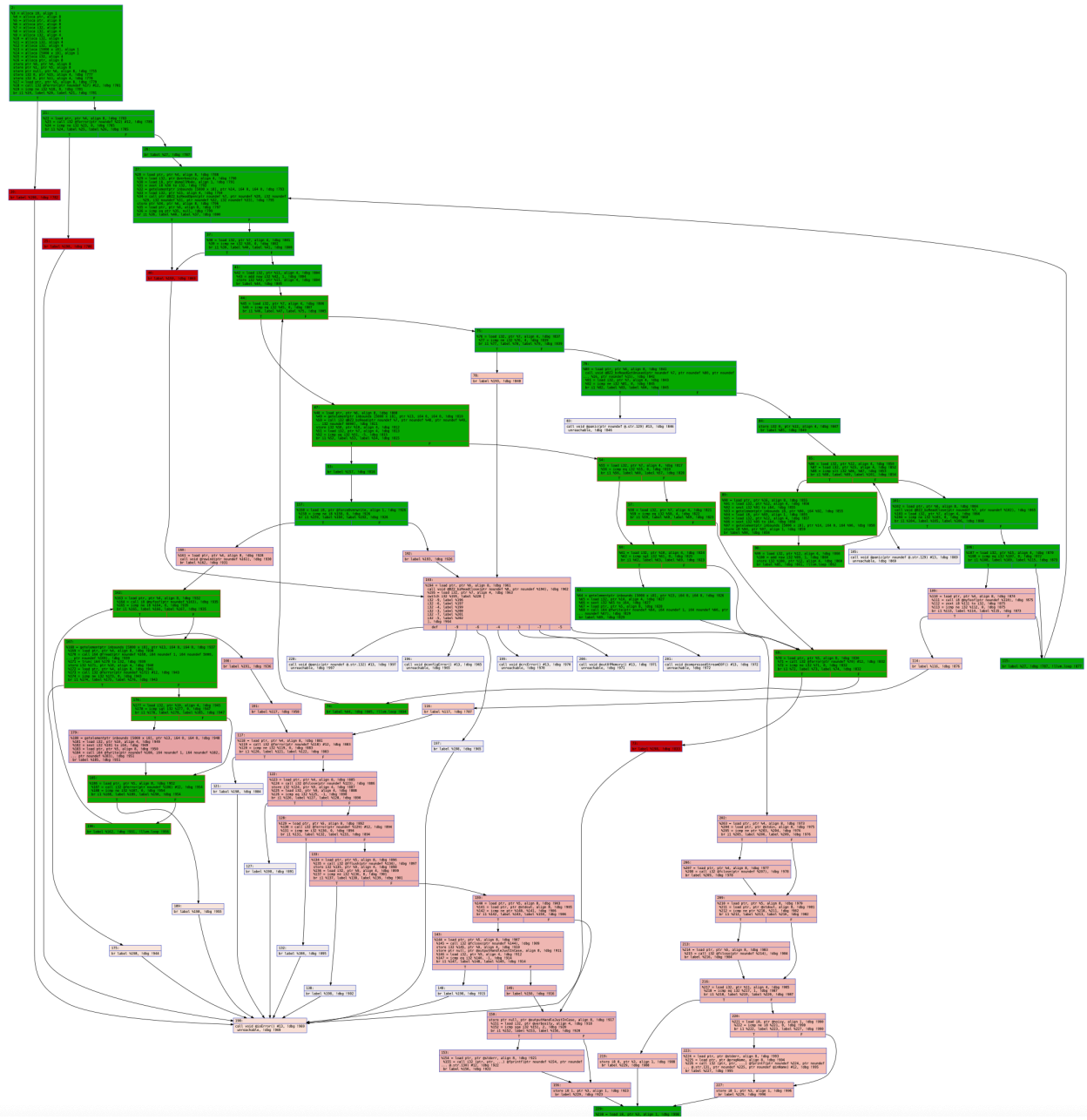
4b.1 Methodology

Coverage data from source-based coverage (Task 3) was mapped onto the CFGs and call graph from Task 1. The LLVM-generated dot files use internal color coding based on execution frequency. These colors were systematically remapped based on coverage data from lcov.

- Green = CFG node covered by the fuzzer
- Red = CFG node not covered by the fuzzer
- Green = function executed by fuzzer (call graph)
- Red = key functions analyzed in report (uncompressStream and BZ2_bzWrite)
- White = function not executed by fuzzer (call graph)

4b.2 Annotated CFG — uncompressStream() (Decompression Campaign)

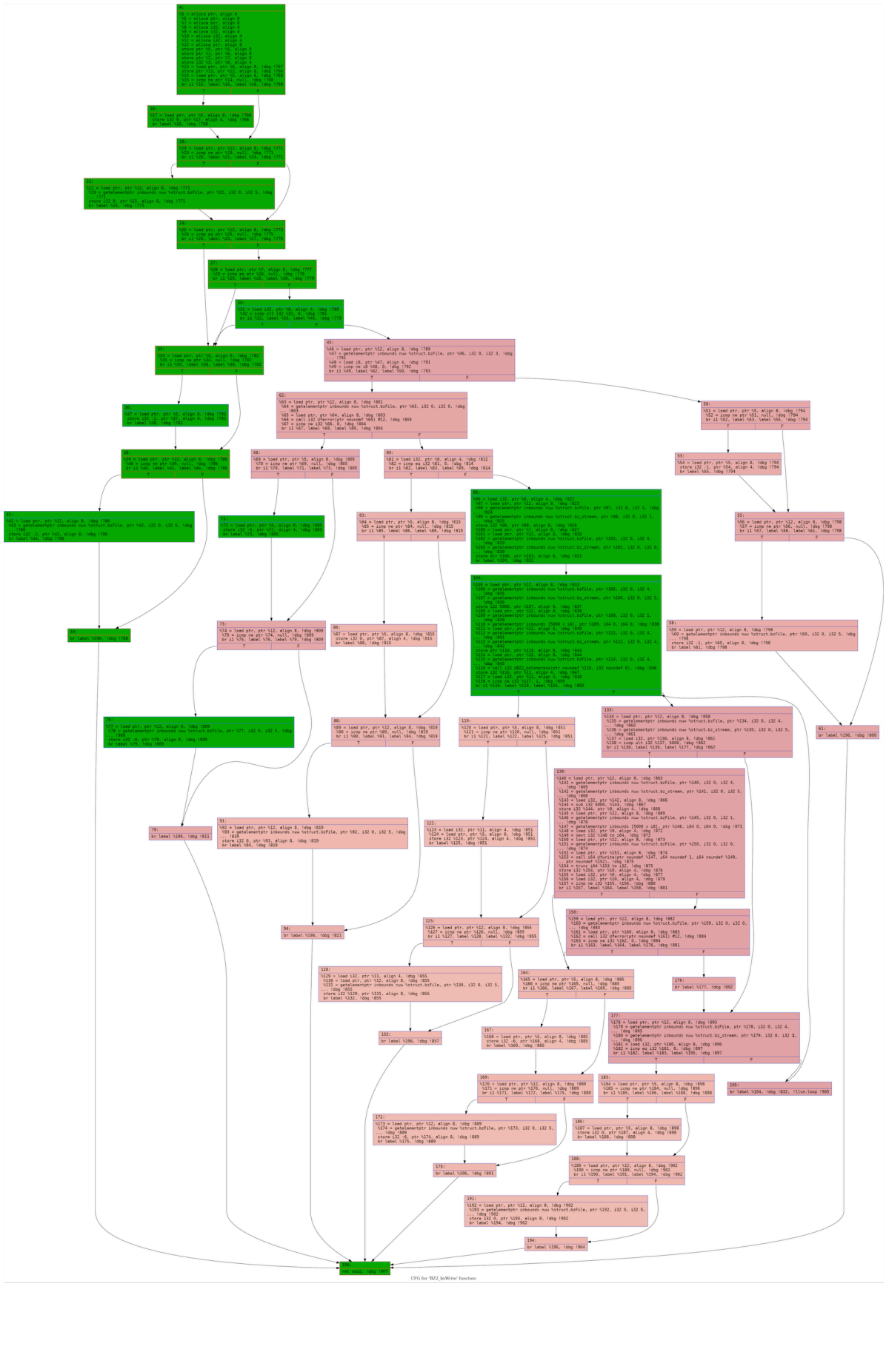
The annotated CFG for uncompressStream() shows coverage results from the decompression campaign. Green nodes represent code paths executed by AFL++, while red nodes indicate unreached paths. The main decompression loop was well covered. The trycat fallback path and outOfMemory handler remained uncovered as they require conditions not generated by input mutation alone.



uncompressStream — Annotated CFG with coverage color

4b.3 Annotated CFG — BZ2_bzWrite() (Compression Campaign)

The annotated CFG for BZ2_bzWrite() shows coverage results from the compression campaign. The main compression path was covered with 8,052 hits. Error return paths (BZ_PARAM_ERROR, BZ_SEQUENCE_ERROR, BZ_IO_ERROR) remained uncovered because AFL++ mutates input data rather than function call parameters.

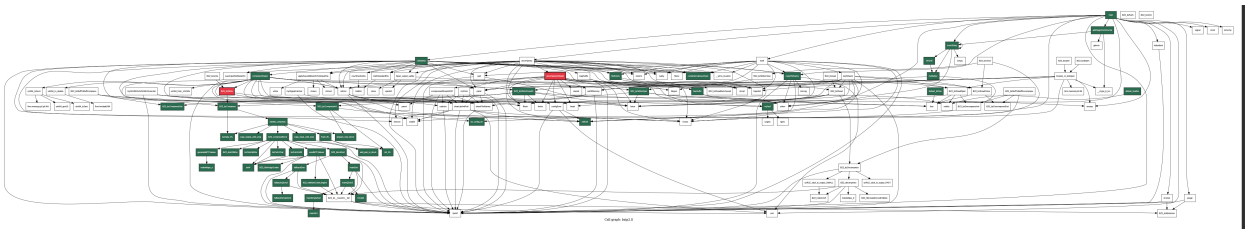


CFG for 'BZ2_bzWrite' function

BZ2_bzWrite — Annotated CFG with coverage colors

4b.4 Annotated Call Graph

The call graph was annotated to show which functions were executed by the fuzzer across both campaigns. Functions executed by either campaign are shown in green, the two key analyzed functions in red, and unexecuted functions in white.



Callgraph — Annotated Call Graph with coverage colors

5. Discussion

5.1 Coverage Patterns

The fuzzing campaigns achieved varying levels of coverage across different parts of bzip2. The decompression campaign covered 29.9% of lines and 30.2% of functions, while the compression campaign achieved higher coverage at 44.6% of lines and 47.2% of functions. The well-covered areas were primarily the main execution paths — the compression and decompression loops, stream handling functions, and error detection routines. These paths were reached easily because the seed files provided valid bzip2 input that guided AFL++ directly into the core processing logic.

The poorly covered areas included error recovery functions, utility functions like uInt64 arithmetic helpers, and signal handler code. These paths require very specific conditions to trigger — for example, the configError() and outOfMemory() functions only execute under resource exhaustion scenarios that are difficult to simulate through input mutation alone.

The program structure significantly influenced coverage results. The nested loop structure in uncompressStream() and BZ2_decompress() was well-explored because AFL++ naturally generates inputs that exercise loop iterations. However, deeply nested conditional branches — particularly those guarding error states — proved difficult to reach. The complex bzip2 file format with its strict magic bytes, block headers, and CRC checksums created a natural barrier.

5.2 Function-Level Coverage

The uncompressStream() function at line 5416 was well-covered by the decompression campaign. The main function body including the BZ2_bzReadOpen() call, the inner read loop, fwrite() calls, and BZ2_bzReadClose() all received thousands of hits. However, several branches remained uncovered: the trycat fallback path and some error handler cases such as BZ_CONFIG_ERROR and BZ_UNEXPECTED_EOF.

The BZ2_bzWrite() function at line 4366 showed 0% coverage in the decompression campaign, which is entirely expected since this function belongs exclusively to the compression API path. In the compression campaign, BZ2_bzWrite() was fully covered with 8,052 hits. The uncovered portions were the error return paths which require invalid parameters or I/O failures that AFL++ did not generate.

5.3 Crash Analysis

Overview

The decompression fuzzing campaign discovered 14 crashes. All were analyzed using AddressSanitizer (ASAN) and all shared the same root cause.

Heap Buffer Overflow	BZ2_decompress()	Line 3369	14 crashes

Vulnerable Code

```
s->tt[s->cftab[uc]] |= (i << 8);
```

Root Cause

The code does not validate that s->cftab[uc] is within the bounds of the s->tt array. When AFL++ mutates the compressed input to corrupt the block header values, s->cftab[uc] becomes

negative, causing the array access to read memory 4 to 8 bytes before the start of the allocated buffer. ASAN confirmed this as a heap buffer underflow.

Call Stack

6 (Entry)	main()	6948
5	uncompress()	6436
4	uncompressStream()	5444
3	BZ2_bzRead()	4601
2	BZ2_bzDecompress()	4244
1 ← CRASH	BZ2_decompress()	3369

Fix

```
if (s->cftab[uc] < 0 || s->cftab[uc] >= nblock) { return BZ_DATA_ERROR; }
```

5.4 Source-Based vs Graph Coverage

Source-based coverage collected via lcov and graph-based coverage derived from CFG annotation provide complementary but distinct perspectives on testing thoroughness. Source coverage measures which executable lines were actually run, giving a precise percentage. Graph coverage maps execution onto CFG nodes and edges, revealing which logical branches and decision points were visited. In our analysis, source coverage proved more immediately useful for identifying untested lines, while graph coverage provided deeper insight into unreachable paths and structural gaps.

5.5 Program Testability and Coverage Barriers

bzip2 presented moderate difficulty for fuzzing. Its command-line interface is straightforward, making it easy to integrate with AFL++. The binary produces deterministic output and executes quickly, allowing AFL++ to achieve execution speeds of up to 5,487 executions per second. However, the bzip2 file format is highly structured, creating significant barriers for coverage. The decompression path requires input that conforms to a strict binary format with magic bytes, block headers, Huffman tables, and CRC checksums.

If we were the developers, several changes would improve testability: exposing individual library functions as separate fuzz targets, adding a fuzzing-friendly mode that accepts raw block data, and modularizing the code into smaller independently testable units.

5.6 Tool Effectiveness and Limitations

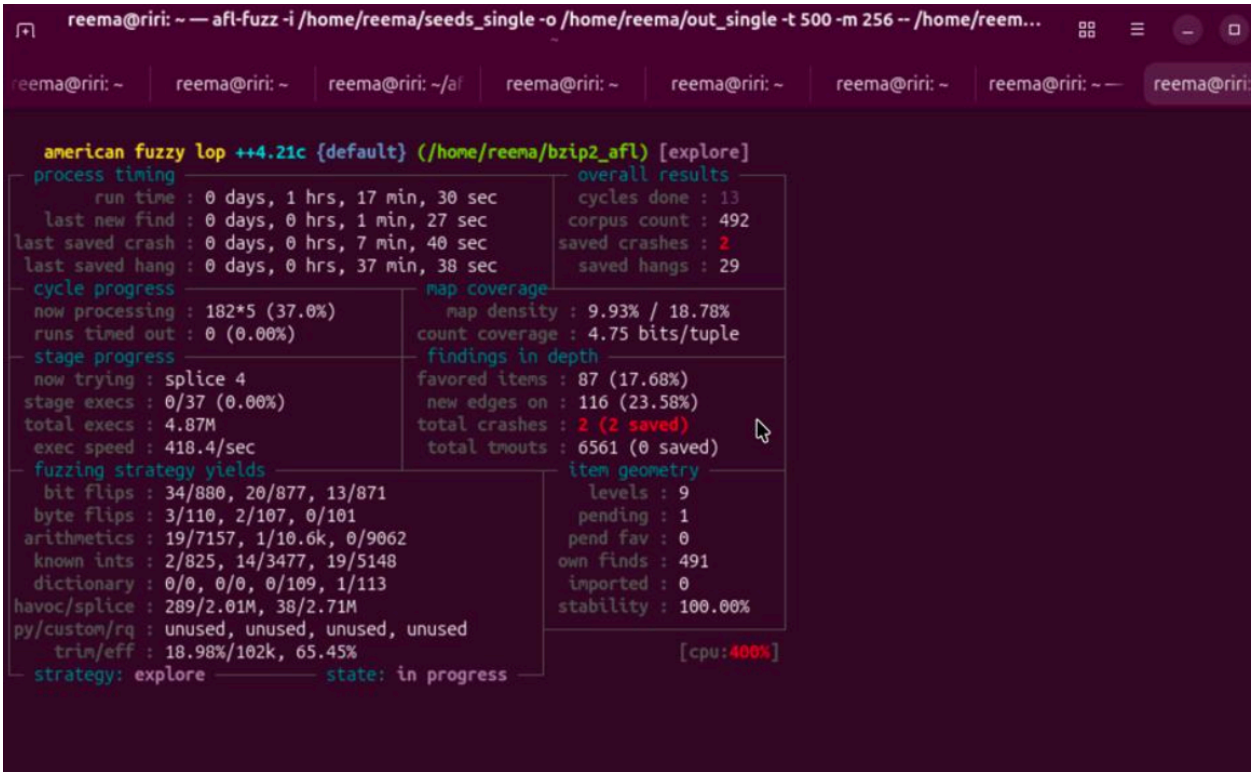
AFL++ proved highly effective at finding diverse inputs and revealing real bugs in bzip2. Within 30 hours of fuzzing the decompression functionality, it discovered 14 crashes and 118 hangs, all rooted in a genuine heap buffer overflow vulnerability. Future campaigns could improve effectiveness by using a custom bzip2-aware mutator, incorporating dictionary-based fuzzing, and running multiple parallel fuzzer instances.

5.7 Seeds and Fuzzing Variability

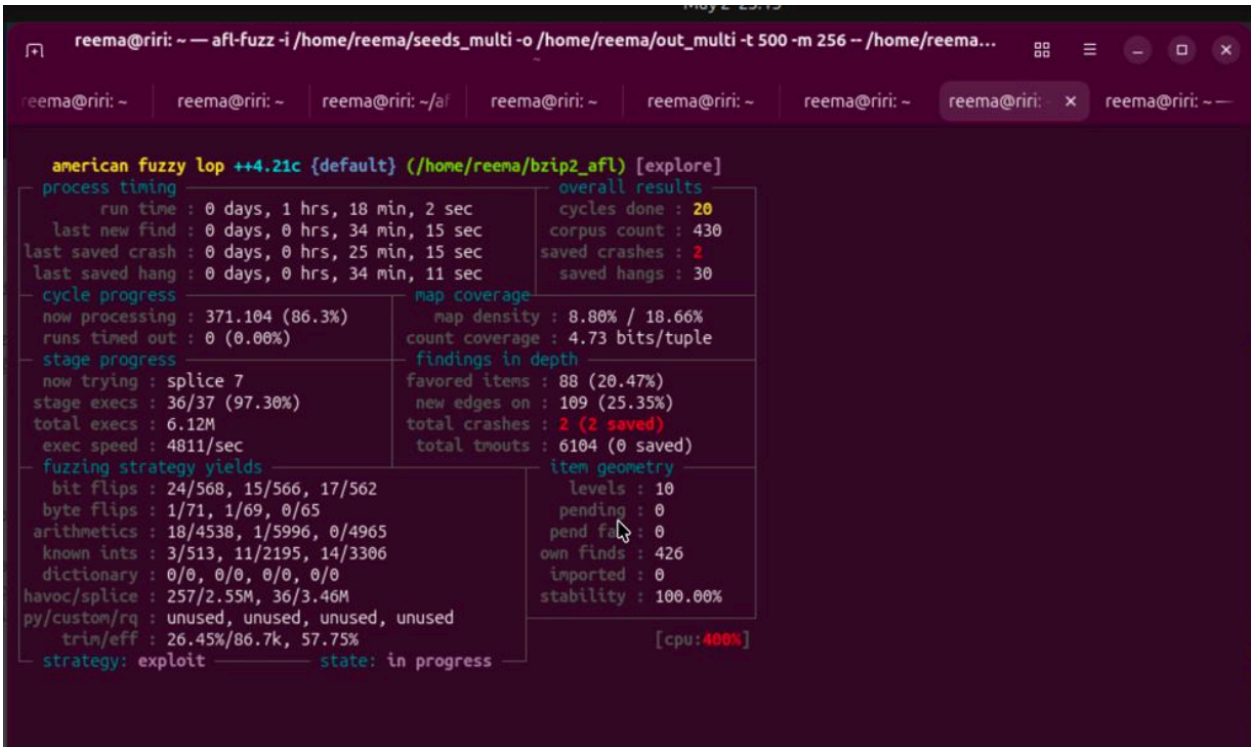
Two one-hour decompression fuzzing campaigns were conducted to evaluate the impact of seed selection. Campaign A used a single minimal seed (letter A compressed with bzip2), while Campaign B used four diverse seeds.

Metric	Campaign A (Single Seed)	Campaign B (Multi Seeds)

Run time	1 hr 17 min	1 hr 18 min
Corpus count	492	430
Saved crashes	2	2
Map density	9.93%	8.80%
Exec speed	418/sec	4,811/sec
New edges	116 (23.58%)	109 (25.35%)



Campaign A (single seed)



Campaign B (multi seeds)

The results revealed a nuanced outcome. Campaign B achieved higher new edges percentage (25.35% vs 23.58%), demonstrating that richer seeds guided AFL++ toward more varied code paths. However, Campaign A generated a larger corpus (492 vs 430). Both campaigns discovered 2 crashes each. These results confirm that diverse seeds provide a meaningful advantage in coverage depth, and this advantage would likely become more pronounced over longer campaign durations.

5.8 Reflections and Lessons Learned

The most surprising aspect of this project was discovering that a mature, widely-used compression utility like bzip2 still contains exploitable memory safety bugs. AFL++ found 14 crashes within hours of starting the decompression campaign. This highlights that manual testing and code review alone are insufficient for finding memory corruption bugs, and automated fuzzing fills a critical gap. Another surprising observation was the significant difference in coverage between compression and decompression — compression achieved nearly 15% more line coverage simply because its input format imposes fewer constraints on the fuzzer.

If we repeated this experiment, we would start fuzzing campaigns earlier, use format-aware seeds from real bzip2 files, and run parallel fuzzer instances. The insights apply directly to real-world software testing: fuzzing should be part of every development pipeline, especially for programs that process untrusted input.